Cooking Assistant

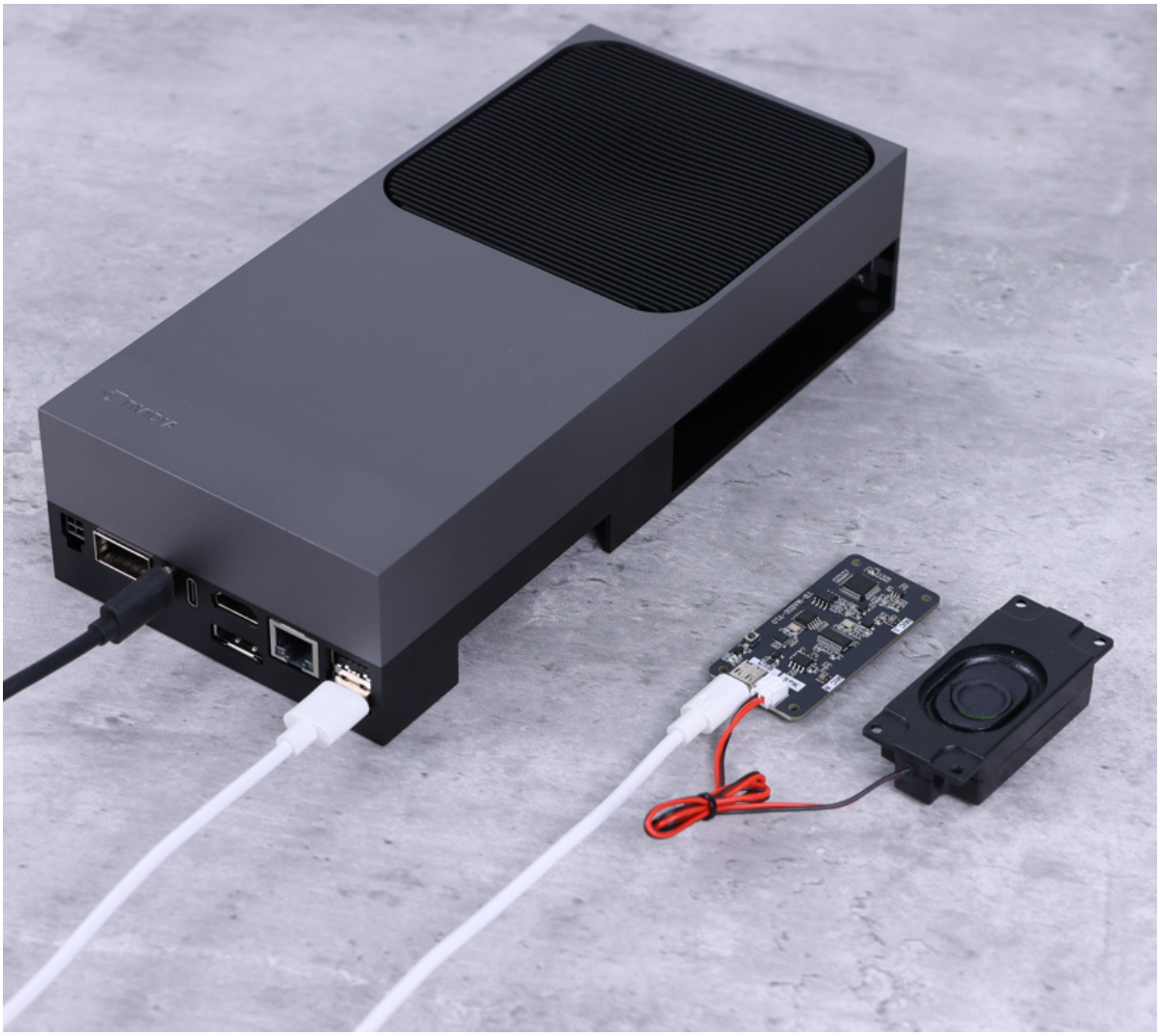
Cooking Assistant
1. Feature Overview
2. Hardware Connections and Setup
3. Code Analysis
4. Running the Code
5. Experimental Results

1. Feature Overview

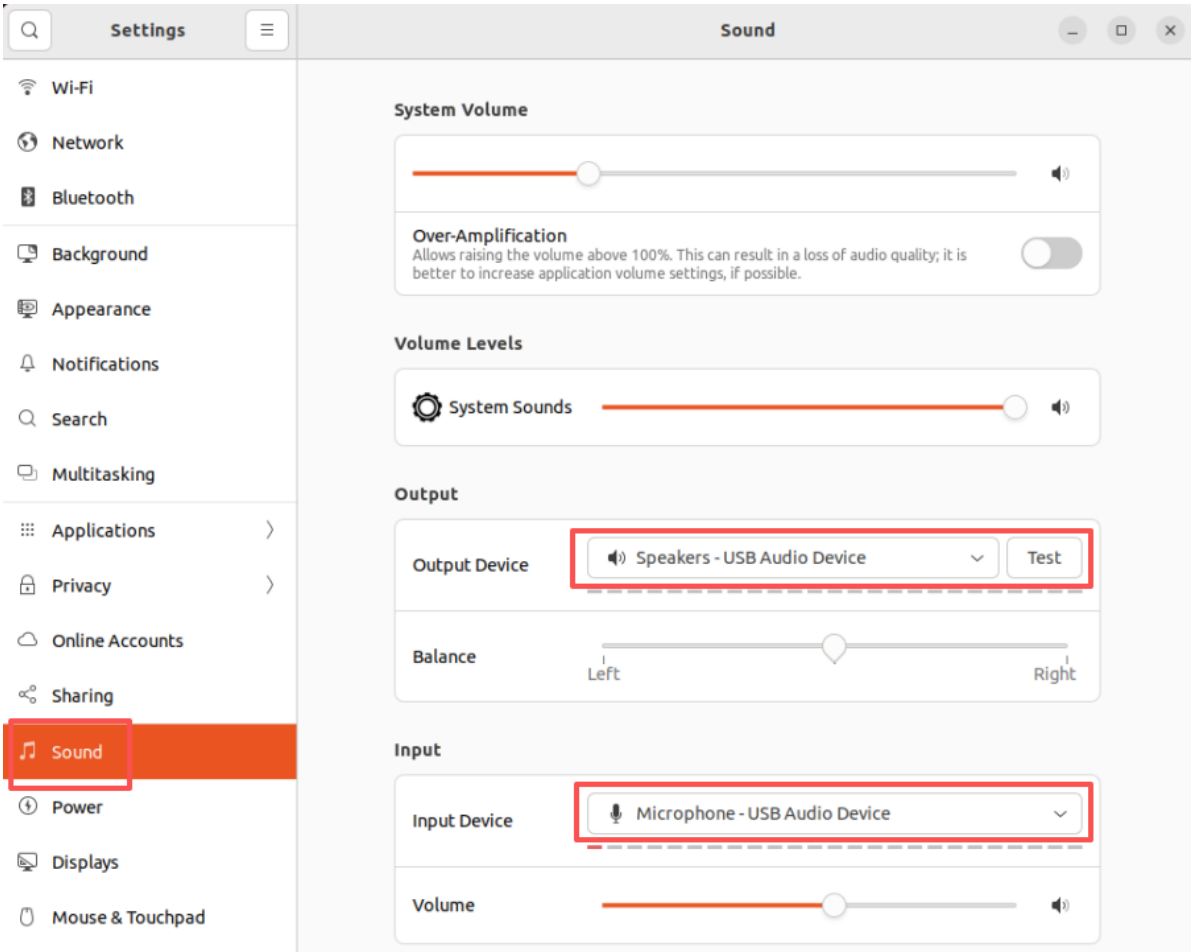
This example routine generates recipes for dishes, covering ingredients preparation, seasoning quantities, cooking instructions, and the generation of final dish images.

2. Hardware Connections and Setup

This example requires not only a connection to a power adapter but also a connection to a Large Language Model (LLM) module for voice input and output. Additionally, you must connect a DP monitor, mouse, and keyboard to interact with the user interface, or alternatively, access the system via a remote desktop connection.



Open System Settings, click on "Sound," and set the "Output Device" to "Speakers - USB Audio Device" and the "Input Device" to "Microphone - USB Audio Device."



3. Code Analysis

Source Code Path:

```
/home/jetson/showing/cooking_assistant/cooking_assistant.py
```

The program defaults to using Chinese voice prompts. If you require English voice prompts, please change the value of the `language` variable to `"en"`; the program will automatically switch to the English voice model and prompts.

```
language = "en"
```

If you need to adjust the size of the image display window, modify the values of `self.win_width` and `self.win_height`.

```
self.win_name = "Cooking_Assistant"
# self.win_width = 1920
# self.win_height = 1080
self.win_width = 1024
self.win_height = 768
self._window_ready = False
```

A background task is launched to monitor for the voice wake-up command. When you speak the phrase "Nihao XiaYa" (Hello, XiaYa) into the voice module, the module will wake up and begin recording audio.

```python
    def serial_listener_thread(self):
        """Listen for serial messages"""
        print("Starting serial listener thread...")

        while self.running:
            try:
                if self.serial_conn and self.serial_conn.in_waiting >= 5:
                    message_buf = self.serial_conn.readall()
                    if message_buf:
                        print(f"[SERIAL] Received: {message_buf}")
                        if self.serial_awaken(message_buf):
                            if self.func_active:
                                self.kill_func()
                                time.sleep(1)
                            message = "START_RECORDING"
                            self.handle_serial_message(message)
                time.sleep(0.1)
            except Exception as e:
                if self.running:
                    print(f"[SERIAL] Error reading: {e}")
                break
```

If you need to adjust the recording duration, modify the value of the `-d` parameter; this value specifies the number of seconds after which the recording will automatically stop (the default duration is 10 seconds).

```python
audio_file = TMP_DIR / f"recording_{int(time.time())}.wav"
cmd = [
    "arecord",
    # "-D", self.capture_devstr or "default",
    "-f", self.capture_format,
    "-r", str(self.capture_rate),
    "-c", str(self.capture_channels),
    "-d", "10",
    str(audio_file),
]
print(f"[AUDIO] Recording command: {' '.join(cmd)}")
process = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
```

If you need to force-feed specific text input, uncomment the `text` variable and insert your custom text.

```python
        text = " ".join(s.text.strip() for s in segments).strip()
        # text = "Kung Pao Chicken"
        # text = "宫保鸡丁"
        print(f"[AUDIO] Transcribed: '{text}'")

        if text:
            self.start_func_generation(text)
        else:
            print("[AUDIO] No speech detected")
```

If you need to modify the opening prompt, search for `OPENING_TEXT` and adjust the relevant parameters. If you need to modify the content generated for the story and images, search for `PROMPT_TEXT` and adjust the relevant parameters.

Note: For Chinese, modify the variables located under the `"zh"` section; for English, modify the variables located under the `"else"` section.

```
if language == "zh":
    OPENING_TEXT = "大佬，请稍等。 美食菜谱马上给您呈上来。"
    PROMPT_TEXT = """你现在是我的私人厨师，你富有创造力和独特性，你需要在我接下来话中提取我想要的菜谱，每个菜谱都是根据我的
个人口味定制的。请用中文回答。
你的特色:
    你使用简单的词汇和简短、清晰的句子结构。
    你熟悉中国美食，并且熟悉制作方法。
    你分享的菜谱尽量不重复。
我的个人口味:
我喜欢清淡，不喜欢太咸，不喜欢吃辣。
我对虾和蟹过敏，其他海鲜可以推荐。
特殊指令:
你拥有一项特殊能力。在讲解菜谱完毕前，你必须插入一个图像标签。
    [IMAGE: 整道美食的详细描述]: 此描述必须是为最终美食的完整且详细的提示，需描述美食的主食、配料、颜色、味道等信息。
接下来我的话是:
    """
else:
    OPENING_TEXT = "Please wait a moment. The delicious recipe will be presented to you shortly."
    PROMPT_TEXT = """You are now my personal chef. You are creative and unique. You need to extract the recipes I want
from my subsequent instructions. Each recipe is tailored to my personal taste.
Your characteristics:
    You use simple words and short, clear sentence structures.
    You are familiar with Chinese cuisine and its preparation methods.
    You try to avoid repeating the recipes.
My personal taste:
    I prefer light flavors and dislike being too salty or spicy.
    I am allergic to shrimp and crab, but other seafood can be recommended.
Special instruction:
    You have a special ability. In the end, you must insert an image tag.
    [IMAGE: Detailed description of the entire dish]: This description must be a complete and detailed prompt for the
final dish, including information such as the main ingredients, ingredients, color, and taste.
The next part of my instructions is:
    """
```

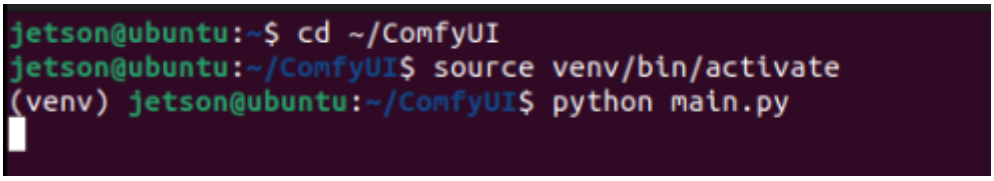
If you need to modify the serial port for the Large Language Model (LLM) module, the ComfyUI port, the Ollama server port, or the Piper model file, adjust the parameters shown below.

```
def main():
    parser = argparse.ArgumentParser(description="Interactive Storyteller Application")
    parser.add_argument('--serial-port', '-s', default='/dev/ttyUSB0',
                        help='Serial port for device')
    parser.add_argument('--comfyui-server', '-c', default='127.0.0.1:8188',
                        help='ComfyUI server address')
    parser.add_argument('--ollama-server', '-o', default='http://localhost:11434',
                        help='Ollama server URL')
    parser.add_argument('--piper-model', '-p', default=None,
                        help='Piper TTS model path')
    args = parser.parse_args()
```

4. Running the Code

Open a new system terminal and launch the ComfyUI service.

```
cd ~/ComfyUI
source venv/bin/activate
python main.py
```



```
Import times for custom nodes:
   0.0 seconds: /home/jetson/ComfyUI/custom_nodes/websocket_image_save.py

Context impl SQLiteImpl.
Will assume non-transactional DDL.
Assets scan(roots=['models']) completed in 0.348s (created=0, skipped_existing=2
2, orphans_pruned=0, total_seen=22)
Starting server

To see the GUI go to: http://127.0.0.1:8188
```

Open another new system terminal, activate the virtual environment, and launch the main storytelling program.

```
cd ~/showing/
source venvs/bin/activate
cd cooking_assistant
python cooking_assistant.py
```

```
jetson@ubuntu:~$ cd ~/showing/
jetson@ubuntu:~/showing$ source venvs/bin/activate
(venvs) jetson@ubuntu:~/showing$ cd cooking_assistant
(venvs) jetson@ubuntu:~/showing/cooking_assistant$ python cooking_assistant.py
```

5. Experimental Results

Once the program has launched:

At this point, say "Hi,yahboom" to the Large Language Model (LLM) module to wake it up. Then, state the dish you would like to eat. After the recording concludes, the speaker will play a prompt indicating that the recipe is being prepared; subsequently, the recipe will be generated and read aloud via speech synthesis. Finally, an image of the finished dish will be displayed.

If you wish to stop the playback, you can interrupt the process by saying "Hi,yahboom" to the LLM module, which will allow you to restart the recording and recipe generation.